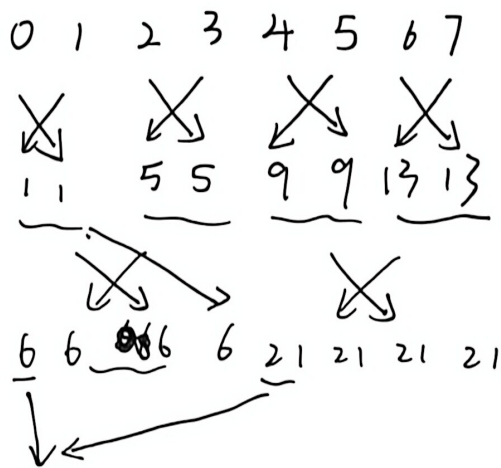


1.



0-1 2-3 4-5 1

0-2 1-3 2

0-4 4

① comm_sz 是 2 的幂

```
#include <stdio.h>
```

```
#include <mpi.h>
```

```
int main( ) {
```

```
    int rank, comm_sz;
```

```
    MPI_Init(NULL, NULL);
```

```
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
```

```
    MPI_Comm_size(MPI_COMM_WORLD, &comm_sz);
```

```
    // 随机初始数为 rank + 1
```

```
    int my_val = rank + 1;
```

```
    int global_sum = my_val;
```

```
    int recv_val = 0;
```

```
    for (int i = 1; i < comm_sz; i *= 2) {
```

```
        int partner = rank ^ i;
```

```
        MPI_Sendrecv(&global_sum, 1, MPI_INT, partner, 0, &recv_val, 1,
```

```
        MPI_INT, partner, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
```

```
        global_sum += recv_val;
```

```
    }
```

```
    MPI_Finalize();
```

```
    return 0; }
```

②任意数量进程：将超过 $2^{\log_2 P}$ 数量进程发送到前 K 个进程，然后就能执行前 $2^{\log_2 P}$ 个进程的蝶式计算，最后再把全局和发送回超出数量的进程。

将①中的 for 循环修改为：

```
int n-size = 1;
while (n-size * 2 <= comm-sz)
    n-size *= 2;

int extra-size = comm-sz - n-size;

if (rank >= n-size) {
    int target = rank - n-size;
    MPI_Send(&global-sum, 1, MPI_INT, target, 0, MPI_COMM_WORLD);
} else if (rank < extra-size) {
    int source = rank + n-size;
    MPI_Recv(&recv-val, 1, MPI_INT, source, 0, MPI_COMM_WORLD);
    global-sum += recv-val;
}

if (rank < n-size) {
    for (int i = 1; i < n-size; i *= 2) {
        int partner = rank ^ i;
        MPI_Sendrecv(&global-sum, 1, MPI_INT, partner, 0, &recv-val,
            1, MPI_INT, partner, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
        global-sum += recv-val;
    }
}

if (rank > n-size) {
    MPI_Recv(&global-sum, 1, MPI_INT, rank - n-size, 1, MPI_COMM_WORLD,
        MPI_STATUS_IGNORE);
} else if (rank < extra-size) {
    MPI_Send(&global-sum, 1, MPI_INT, rank + n-size, 1, MPI_COMM_WORLD);
}
```

2:

① clock 函数在代码累积的 CPU 运算时间达到 1 个时钟周期才会给出非零运行时间, 即达到 1 纳秒

② MPI.Wtime 是“墙上时间”, 衡量真实世界的物理时间,

当进程 A 发送消息并等待进程 B 的回复时, 消息在网络中的传输延时也会被记录, 所以 Wtime 反映整个通信过程的开销

Clock 是 CPU 时间, 只统计进程在 CPU 实际执行指令时间, 在 MPI 阻塞接收期间, 当前进程被挂起时不占用 CPU, 所以 clock 不会记录网络传输的延时时间, 它只记录系统调用和数据打包/解包时间

3.

块分布

1, 4, 7 $\rightarrow$ P1

循环分布

P0 0, 1, 2, $\rightarrow$ P0 0 3 6

P1 3, 4, 5 $\leftarrow$ P1 1 4 7

P2 6, 7, 8 $\xrightarrow{1 \rightarrow P0}$ P2 2 5 8
 $\xrightarrow{4 \rightarrow P1}$
 $\xrightarrow{7 \rightarrow P2}$

假设数组 N 大小能被 P^2 整除, 这样每个进程发送给其他进程的数据量都相等

1) 块分布 $\rightarrow$ 循环分布: 进程 rank 有数据 $[rank * n, (rank+1) * n]$

对索引为 x 的元素对应目标进程 $x \% P$, 所以每个进程只用遍历本地数据, 计算全局索引, 打包起来, 最后发送到目标进程

这种每个进程都要向通信域内所有进程发送且同时接收一段数据
可以使用 MPI.Alltoall (send-buf, send-count, send-type, recv 同理,
 $\downarrow$
发送给每一个目标进程数据量 MPI.Comm.Work

2) 循环分布 $\rightarrow$ 块分布: 进程 rank 有数据 rank, rank+p, rank+2p, ...
全局索引 g 的元素, 对应目标进程 g/n , 操作与块到循环同理


```

#include <stdio.h>
#include <stdlib.h>
#include <mpi.h>

int main(int argc, char **argv)
{
    int rank, p;
    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &p);

    // 假设全局大小 N 是  $p^2$  的 100000 倍, 确保整除, 方便使用 MPI_Alltoall
    long long N = (long long)p * p * 100000;
    long long local_n = N / p; // 每个进程拥有的总数据量
    long long chunk = local_n / p; // 每次发给单个目标进程的数据量

    // 分配内存
    int *block_data = (int *)malloc(local_n * sizeof(int));
    int *cyclic_data = (int *)malloc(local_n * sizeof(int));
    int *send_buf = (int *)malloc(local_n * sizeof(int));
    int *recv_buf = (int *)malloc(local_n * sizeof(int));
    int *counts = (int *)malloc(p * sizeof(int));

    // 1. 初始化块分布数据
    for (long long i = 0; i < local_n; i++)
    {
        block_data[i] = rank;
    }

    // 测试 1: 从块分布转换为循环分布
    MPI_Barrier(MPI_COMM_WORLD);
    double start_b2c = MPI_Wtime();

    // 1a. 打包数据
    for (int i = 0; i < p; i++)
        counts[i] = 0; // 重置计数器
    for (long long i = 0; i < local_n; i++)
    {
        long long g = rank * local_n + i; // 计算全局索引
        int dest = g % p; // 循环分布的目标进程
        // 将数据放入对应目标进程的发送槽位
        send_buf[dest * chunk + counts[dest]] = block_data[i];
        counts[dest]++;
    }

    // 1b. 全局全交换
    MPI_Alltoall(send_buf, chunk, MPI_INT, recv_buf, chunk, MPI_INT, MPI_COMM_WORLD);

    // 1c. 解包数据
    for (long long i = 0; i < local_n; i++)
    {
        cyclic_data[i] = recv_buf[i];
    }

    MPI_Barrier(MPI_COMM_WORLD);
    double end_b2c = MPI_Wtime();
    double time_b2c = end_b2c - start_b2c;

```

```

// 测试 2: 从 循环分布转换为块分布
MPI_Barrier(MPI_COMM_WORLD);
double start_c2b = MPI_Wtime();

// 2a. 打包数据
for (int i = 0; i < p; i++)
    counts[i] = 0;
for (long long i = 0; i < local_n; i++)
{
    long long g = rank + i * p; // 循环数据的全局索引
    int dest = g / local_n;     // 块分布的目标进程
    send_buf[dest * chunk + counts[dest]] = cyclic_data[i];
    counts[dest]++;
}

// 2b. 全局全交换
MPI_Alltoall(send_buf, chunk, MPI_INT, recv_buf, chunk, MPI_INT, MPI_COMM_WORLD);

// 2c. 解包数据
for (long long i = 0; i < local_n; i++)
{
    block_data[i] = recv_buf[i];
}

MPI_Barrier(MPI_COMM_WORLD);
double end_c2b = MPI_Wtime();
double time_c2b = end_c2b - start_c2b;

// 结果统计与输出 (取所有进程耗时的最大值)
double max_time_b2c, max_time_c2b;
MPI_Reduce(&time_b2c, &max_time_b2c, 1, MPI_DOUBLE, MPI_MAX, 0, MPI_COMM_WORLD);
MPI_Reduce(&time_c2b, &max_time_c2b, 1, MPI_DOUBLE, MPI_MAX, 0, MPI_COMM_WORLD);

if (rank == 0)
{
    printf("总数据量 N = %lld 整数 (约 %lld MB)\n", N, N * sizeof(int) / 1024 / 1024);
    printf("进程数量 P = %d\n", p);
    printf("-----\n");
    printf("块分布 -> 循环分布耗时: %f 秒\n", max_time_b2c);
    printf("循环分布 -> 块分布耗时: %f 秒\n", max_time_c2b);
    printf("-----\n");
}

free(block_data);
free(cyclic_data);
free(send_buf);
free(recv_buf);
free(counts);

MPI_Finalize();
return 0;
}

```

总数据量 N = 40000000 整数 (约 15 MB)

进程数量 P = 2

块分布 -> 循环分布 (Block to Cyclic) 耗时: 0.092022 秒

循环分布 -> 块分布 (Cyclic to Block) 耗时: 0.070692 秒

3.A

① 初始数据分布

$$\begin{array}{l} A: \\ P0 \left(\begin{array}{ccc} 1 & 4 & 7 \end{array} \right) \\ P1 \left(\begin{array}{ccc} 2 & 5 & 8 \end{array} \right) \\ P2 \left(\begin{array}{ccc} 3 & 6 & 9 \end{array} \right) \end{array} \quad B: \left(\begin{array}{ccc} -1 & -2 & -3 \\ -4 & -5 & -6 \\ -7 & -8 & -9 \end{array} \right)$$

② 初始对齐:

A 第 i 行左移 i 步

B 第 j 列上移 j 步

$$A^{(0)} = \left(\begin{array}{ccc} 1 & 4 & 7 \\ 5 & 8 & 2 \\ 9 & 3 & 6 \end{array} \right) \quad B^{(0)} = \left(\begin{array}{ccc} -1 & -5 & -9 \\ -4 & -8 & -3 \\ -7 & -2 & -6 \end{array} \right)$$

③ 乘加和循环移位

$K=0$

$$1) C^{(1)} = C^{(0)} + A^{(0)} \circ B^{(0)}$$

$$C^{(1)} = \left(\begin{array}{ccc} -1 & -20 & -63 \\ -20 & -64 & -6 \\ -63 & -6 & -36 \end{array} \right)$$

2) $A^{(0)}$ 每行左移 1 步

$B^{(0)}$ 每列上移 1 步

$$A^{(1)} = \left(\begin{array}{ccc} 4 & 7 & 1 \\ 8 & 2 & 5 \\ 3 & 6 & 9 \end{array} \right) \quad B^{(1)} = \left(\begin{array}{ccc} -4 & -8 & -3 \\ -7 & -2 & -6 \\ -1 & -5 & -9 \end{array} \right)$$

$K=1$

$$C^{(2)} = C^{(1)} + A^{(1)} \circ B^{(1)}$$

$$1) C^{(2)} = \left(\begin{array}{ccc} -17 & -76 & -66 \\ -76 & -68 & -36 \\ -66 & -36 & -117 \end{array} \right)$$

$$2) A^{(2)} = \left(\begin{array}{ccc} 7 & 1 & 4 \\ 2 & 5 & 8 \\ 6 & 9 & 3 \end{array} \right) \quad B^{(2)} = \left(\begin{array}{ccc} -7 & -2 & -6 \\ -1 & -5 & -9 \\ -4 & -8 & -3 \end{array} \right)$$

$K=2$

$$C^{(3)} = C^{(2)} + A^{(2)} \circ B^{(2)}$$

$$1) C^{(3)} = \left(\begin{array}{ccc} -49 & -2 & -24 \\ -2 & -25 & -72 \\ -24 & -72 & -9 \end{array} \right) + C^{(2)} = \left(\begin{array}{ccc} -66 & -78 & -90 \\ -78 & -93 & -108 \\ -90 & -108 & -126 \end{array} \right)$$

$C^{(3)}$ 为最终结果

3.3

1) 局部排序: 每个处理器有 n/p 个数据, 调用串行排序算法

$$T_1 = O\left(\frac{n}{p} \log \frac{n}{p}\right)$$

2) 奇偶交换 p 轮: 依次互相发送数据 n/p 个, 需要 $t_d + t_s \cdot \frac{n}{p}$

~~总~~ $T_2 = p(t_d + t_s \frac{n}{p}) = pt_d + nts$
通信时间:

归并交换数据时间: $T_3 = p \cdot O\left(\frac{n}{p}\right) = O(n)$

并行算法执行时间 $T_p = T_1 + T_2 + T_3 = O\left(\frac{n}{p} \log \frac{n}{p}\right) + pt_d + nts + O(n)$

加速比 $\odot S = \frac{T_s}{T_p} = \frac{O(n \log n)}{O\left(\frac{n}{p} \log \frac{n}{p}\right) + pt_d + \odot nts + O(n)}$